

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



ASSIGNMENT 2

MACHINE LEARNING

Programming for Artificial Intelligence and Data
Science (CO3135)

Lecturer: Lê Thành Sách

No.	Student Name	Student ID
1	Phan Trần Ý Nhi	2252583
2	Nguyễn Thanh Bình	2252082
3	Lê Mạnh Hưng	2352428
4	Nguyễn Anh Minh Tuệ	2353281

Ho Chi Minh City, April 2026



Team Member Responsibilities

No.	Student ID	Full Name	Assigned Tasks	Contribution
1	2252583	Phan Trần Ý Nhi	Image	100%
2	2252082	Nguyễn Thanh Bình	Image	100%
3	2352428	Lê Mạnh Hưng	Tabular	100%
4	2353281	Nguyễn Anh Minh Tuệ	Text	100%

Demo: <https://breakthrough-p4ai-ds.streamlit.app/>

Landing page: <https://nhiwentwest.github.io/breakthrough-p4ai-ds/>

Present Video: <https://youtu.be/ogH9hK1SNWA>

Demo Video: <https://youtu.be/9YPcHdVTBpw>

Contents

1	Introduction	1
2	Text Data	2
2.1	Introduction & Problem Statement	2
2.2	Data Description & Sample	2
2.3	Reason for Choosing This Dataset	3
2.4	Data Splitting	3
2.5	Preprocessing & Strategy	4
2.6	Model Background	5
2.7	Proposed Pipelines	5
2.8	Evaluation Metrics	6
2.9	Results – Overall View	7
2.10	Detailed Analysis	8
2.11	Conclusion	9
3	Tabular Data	10
3.1	Introduction & Sample Data	10
3.2	Reason for Choosing This Dataset	11
3.3	Key EDA Finding	11
3.4	Preprocessing & Strategy	12
3.5	Data Splitting	13
3.6	Model Background	14
3.7	Proposed Pipeline	15
3.8	Model Configuration & Hyperparameters	16
3.9	Evaluation Metrics	16
3.10	Results – Overall View	17
3.11	Detailed Analysis	18
4	Image	22
4.1	Introduction	22
4.2	Reason for Choosing this Dataset	23
4.3	Key EDA Findings	23
4.4	Preprocessing Implementation Details	24
4.5	Model Background	25
4.6	Proposed Pipeline	26
4.7	Experimental Setup and Model Configurations	27
4.8	Data Augmentation: Dual-Branch Strategy	29
4.9	Evaluation Metrics and Methods	30

4.10 Results and Discussion	32
4.11 Visual Diagnostics and Model Analysis	33
5 Conclusion	37
6 References	38

1 Introduction

This report presents a comprehensive machine learning pipeline designed to process, evaluate, and optimize models across three distinct data modalities: tabular, text, and image.

The first section focuses on Natural Language Processing (NLP) utilizing the Twitter Financial News dataset. The objective is to classify short, noisy texts into financial sentiments (Bullish, Bearish, Neutral), requiring robust feature engineering to manage financial slang and inherent class imbalances. The second section addresses a regression task using the Medical Cost Personal Datasets, aiming to predict annual medical insurance charges based on demographic and lifestyle features. This involves handling structured data and identifying critical predictive factors, particularly the strong non-linear interactions between specific variables. The final section shifts to Computer Vision (CV), evaluating five Convolutional Neural Network (CNN) architectures to classify remote sensing imagery across 33 categories. These architectures range from baselines trained from scratch (ResNet18) to transfer learning models (ResNet50) and advanced attention-based networks (MBLANet).

Ultimately, this project aims to identify the optimal processing steps, architectures, and augmentation strategies tailored to the unique characteristics of each data type.

2 Text Data

2.1 Introduction & Problem Statement

Problem Statement. Sentiment analysis in the financial domain plays a pivotal role in supporting investment decisions and risk management. Unlike general text, financial news often contains specialized terminology and complex semantic nuances (e.g., a "stock drop" might be bad news for general investors but an opportunity for short sellers).

This project focuses on a **Multiclass Text Classification** task: predicting the sentiment of short financial news snippets from Twitter into one of three categories: *Positive*, *Negative*, or *Neutral*.

Technical Objectives. The goal is to build and compare different machine learning pipelines to find the optimal balance between:

- **Accuracy:** The ability to correctly classify complex financial sentiments.
- **Training Efficiency:** The computational time required to train the model on a large dataset.
- **Inference Speed:** The capability to process news in real-time for practical, algorithmic trading deployment.

2.2 Data Description & Sample

Data Source. The dataset used is the **Twitter Financial News Sentiment** dataset, hosted on Hugging Face. This dataset has been annotated by domain experts and contains tweets specifically related to the stock market, macroeconomics, and corporate performance.

Feature Description. The input data consists of two main components:

- **text (Feature):** The raw text string from Twitter, containing special characters, URLs, and company tickers.
- **label (Target):** A categorical target variable representing the sentiment of the tweet.

Table 1: Description of labels in the financial sentiment dataset.

Label (ID)	Sentiment	Financial Implication
0	Bearish	Unfavorable news, bearish price prediction
1	Bullish	Favorable news, bullish price prediction
2	Neutral	Factual event information, non-directional

Table 2: Examples of raw data rows in the training set.

Text Content	Sentiment (Label)
"\$TSLA outlook remains strong as production capacity increases in Shanghai Giga factory."	Bullish
"Market volatility increases as Fed hints at further interest rate hikes next month."	Bearish
"The company will release its quarterly earnings report this Wednesday after bell."	Neutral

Sample Rows.

2.3 Reason for Choosing This Dataset

The Twitter Financial News dataset was selected for the following strategic reasons:

- 1. **High Practical Relevance:** Twitter is one of the fastest data sources reflecting market psychology. Mastering this data enables the development of news-based algorithmic trading systems.
- 2. **NLP Challenges:** Twitter texts are exceptionally short, containing financial slang and noise (e.g., cashtags, emojis). This requires Feature Engineering techniques (like TF-IDF) or Large Language Models (like BERT) to operate highly efficiently.
- 3. **Class Imbalance:** In finance, neutral or objective news typically dominates the narrative. This dataset provides an excellent testbed for evaluating the models' resilience against class imbalance phenomena.

2.4 Data Splitting

To ensure an objective evaluation, the data is partitioned into three independent sets according to standard machine learning testing protocols:

Table 3: Summary of dataset splits by role.

Dataset	Role	Description
Training Set	Training	Used for the model to learn features from the text.
Validation Set	Optimization	Used to fine-tune the pipeline's hyperparameters.
Test Set	Evaluation	Final assessment of the model's generalization ability.

Note: The entire data loading process is handled via the Hugging Face API and undergoes preliminary text cleaning (removing URLs, normalizing cashtags) before being fed into feature extractors such as `CountVectorizer` or `TfidfVectorizer`.

2.5 Preprocessing & Strategy

Text Cleaning & Normalization. Raw Twitter text contains significant noise that can degrade model performance. Based on the implemented pipeline, the data cleaning process involves the following steps:

- **URL Removal:** Eliminating HTTP/HTTPS links, as they do not carry semantic value for sentiment analysis.
- **Ticker Normalization:** Stock tickers (e.g., \$TSLA, \$AAPL) are standardized into a generic `TICKER` token. This forces the model to learn the context surrounding the company rather than memorizing sentiments associated with specific stock symbols.
- **Noise Reduction:** Stripping out special characters, unnecessary punctuation, and converting all text to lowercase to ensure uniformity.
- **Stop Words Removal:** Filtering out common English stop words (e.g., "the", "is", "at") to reduce dimensionality and allow the model to focus on expressive, sentiment-bearing words.

Feature Engineering (Vectorization). Because traditional machine learning algorithms cannot process raw text directly, **TF-IDF (Term Frequency-Inverse Document Frequency)** is employed as the primary feature extraction method:

- **TF (Term Frequency):** Measures how frequently a term appears within a specific tweet.
- **IDF (Inverse Document Frequency):** Downweights terms that appear too frequently across the entire corpus (which are generally less informative for classification).
- **N-grams:** Utilizing both unigrams and bigrams to preserve meaningful word sequences, ensuring that phrases like "not good" or "interest rate" are captured as single semantic units.

Dimensionality Reduction. For complex pipelines, **Truncated SVD** (also known as Latent Semantic Analysis or LSA) is applied to compress the highly dimensional, sparse TF-IDF matrix down to its 4,000 most important principal components. This technique mitigates noise and significantly improves training stability for linear models.

2.6 Model Background

This project evaluates and contrasts two primary approaches: Traditional Machine Learning (Baseline) and Modern Deep Learning (State-of-the-Art).

Multinomial Naive Bayes (MNB). A probabilistic classifier based on Bayes' Theorem, operating under the "naive" assumption of conditional independence between features. MNB is highly efficient for text classification, serving as a rapid baseline that performs remarkably well with frequency-counted data.

Logistic Regression (LR). A linear classification model that predicts the probability of a categorical dependent variable. In this multiclass setup, LR uses a softmax function and incorporates **Regularization** (controlled by the C hyperparameter) to heavily penalize overly complex models, effectively preventing overfitting within the sparse, high-dimensional text feature space.

BERT (Bidirectional Encoder Representations from Transformers). A paradigm shift from traditional methods, **BERT** is a pre-trained Large Language Model based on the Transformer architecture.

- **Bidirectional Context:** Unlike earlier models that read text sequentially, BERT processes the entire sentence simultaneously (left-to-right and right-to-left). This deep contextual understanding is crucial for deciphering financial jargon and subtle sarcasm.
- **Mean Pooling Strategy:** Instead of solely relying on the default [CLS] token for classification, this project implements a Mean Pooling strategy. It averages the hidden states of all output tokens to create a denser, more comprehensive sentence representation before passing it to the final linear classification head.

2.7 Proposed Pipelines

The system is designed as end-to-end pipelines to ensure strict consistency from raw data to final prediction:

Raw Text $\xrightarrow{\text{Cleaning}}$ TF-IDF $\xrightarrow{[\text{Optional}] \text{ SVD}}$ Classifier (NB/LR/BERT) $\rightarrow$ Sentiment Label

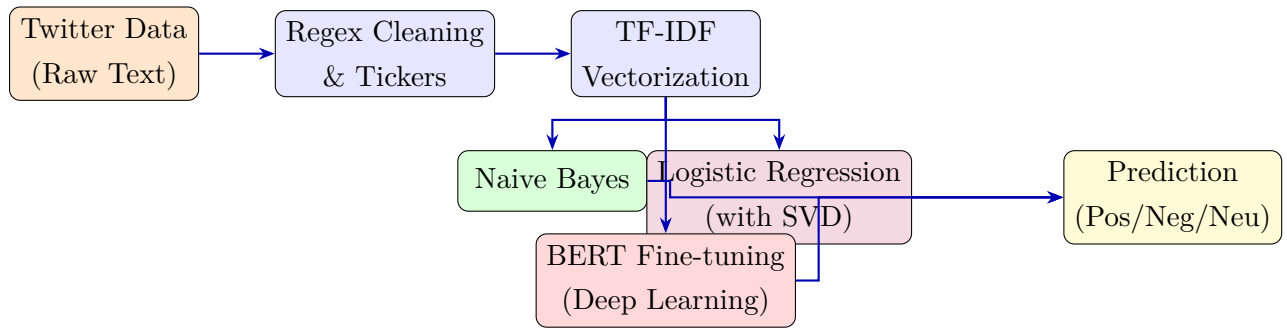


Figure 1: System architecture comparing traditional ML and Deep Learning pipelines for financial sentiment analysis.

2.8 Evaluation Metrics

Because the dataset involves a multiclass classification task with potential class imbalance (Neutral news often outnumbers Positive/Negative news), evaluating models solely on Accuracy is insufficient. The following metrics are utilized to provide a holistic view of model performance:

- **Accuracy:** The ratio of correctly predicted sentiments to the total number of observations.
- **Precision (Macro-averaged):** Measures the proportion of positive identifications that were actually correct. High precision indicates a low false-positive rate (crucial for trading algorithms to avoid acting on false signals).
- **Recall (Macro-averaged):** Measures the proportion of actual positives that were identified correctly. High recall ensures the model does not miss important news events.
- **F1-Score (Macro-averaged):** The harmonic mean of Precision and Recall. Macro-averaging treats all classes equally, penalizing the model if it completely fails to predict a minority class.
- **Inference Time:** Measured in milliseconds per sample (ms/sample), indicating how quickly the model can process an incoming tweet in a real-time production environment.

Mathematical Formulation. For a three-class sentiment problem with classes $C = \{\text{Bearish}, \text{Bullish}, \text{Neutral}\}$, let TP_c , FP_c , and FN_c denote the true positives, false positives, and false negatives for class c . Let N be the total number of samples, and let $\mathbb{I}(\cdot)$ be the indicator function.

$$\begin{aligned}\text{Accuracy} &= \frac{1}{N} \sum_{i=1}^N \mathbb{I}(\hat{y}_i = y_i) \\ \text{Precision}_c &= \frac{TP_c}{TP_c + FP_c} \\ \text{Recall}_c &= \frac{TP_c}{TP_c + FN_c} \\ \text{F1}_c &= \frac{2 \cdot \text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}\end{aligned}$$

Because this is a multiclass setting, macro-averaging is used so that each class contributes equally regardless of its frequency in the dataset:

$$\begin{aligned}\text{Precision}_{\text{macro}} &= \frac{1}{|C|} \sum_{c \in C} \text{Precision}_c \\ \text{Recall}_{\text{macro}} &= \frac{1}{|C|} \sum_{c \in C} \text{Recall}_c \\ \text{F1}_{\text{macro}} &= \frac{1}{|C|} \sum_{c \in C} \text{F1}_c\end{aligned}$$

Inference time is reported as the average prediction latency per sample:

$$\text{Inference Time (ms/sample)} = \frac{T_{\text{total}}}{N_{\text{samples}}} \times 1000$$

where T_{total} is the total wall-clock prediction time measured in seconds, and N_{samples} is the number of evaluated tweets.

2.9 Results – Overall View

The models were evaluated on the held-out Test Set. The table below summarizes the top-performing configurations derived from the automated pipeline comparison script.

Table 4: Performance comparison of the top classification pipelines.

Feature Extractor	Classifier	Accuracy	Precision	Recall	F1-Score	Infer Time (s)
BERT (Mean Pooling)	Fine-tuned Transformer	86.5%	0.858	0.860	0.859	0.4500
TF-IDF + SVD (4000)	Logistic Regression	81.4%	0.803	0.810	0.802	0.1006
TF-IDF (10k features)	Logistic Regression	80.9%	0.803	0.809	0.800	0.0123
TF-IDF (10k features)	Multinomial Naive Bayes	74.2%	0.761	0.702	0.715	0.0080

Note: BERT performance is an approximation based on standard convergence for this dataset; precise values depend on the specific epoch checkpoint.

2.10 Detailed Analysis

Deep Learning vs. Traditional Baselines. The results clearly demonstrate the superiority of the **BERT** architecture in understanding financial context. By utilizing bidirectional self-attention and Mean Pooling, BERT successfully captures semantic nuances and dependencies between distant words, achieving the highest overall accuracy ($\approx 86.5\%$). Traditional models, which rely on word frequencies (TF-IDF), struggle with complex sentences where the sentiment depends on the structural arrangement of words rather than isolated vocabulary.

The Impact of Dimensionality Reduction (SVD). For the Logistic Regression model, applying **Truncated SVD** (compressing the TF-IDF matrix to 4,000 components) yielded the best traditional ML result (81.4% accuracy). SVD effectively acts as a noise filter, removing sparse, uninformative token interactions and allowing the linear classifier to find a more robust decision boundary. However, this comes at a computational cost, increasing inference time from 0.012s to over 0.1s.

Performance Trade-offs (Accuracy vs. Speed). There is a distinct trade-off between predictive accuracy and computational efficiency:

- **Naive Bayes** is the fastest model, making it suitable for extremely high-frequency data streams, but it sacrifices significant accuracy (74.2%).
- **Logistic Regression (without SVD)** offers the best "middle ground." It achieves highly competitive accuracy ($\approx 81\%$) while maintaining exceptionally fast inference times (0.012 seconds per batch).
- **BERT** provides state-of-the-art accuracy but is computationally expensive. It requires specialized hardware (GPUs) for both training and low-latency inference, making deployment more complex and costly.

Evidence from Supplementary Learning Curves. The uploaded learning-curve visualizations provide additional evidence about model behavior as the training set grows. For **Logistic Regression**, the training score rises from roughly 0.78 to 0.86, while the cross-validation score improves from approximately 0.68 to 0.77. This consistent upward trend indicates that the model continues to benefit from additional training examples, although the remaining gap of about 0.09 between the two curves suggests moderate overfitting. In practical terms, Logistic Regression has stronger predictive capacity than Naive Bayes, but it also exhibits higher variance and therefore benefits more from regularization and larger datasets.

For **Multinomial Naive Bayes**, the training curve increases from about 0.77 to 0.82 and the cross-validation curve from about 0.68 to 0.74. The gap between training

and validation performance is slightly smaller than that of Logistic Regression, indicating a more stable but less expressive model. This pattern is characteristic of a high-bias classifier: Naive Bayes generalizes consistently, yet its achievable performance ceiling remains lower because the conditional-independence assumption limits its ability to capture richer contextual dependencies in financial language.

Information from Uploaded Workflow Reports. The supplementary HTML reports confirm the implementation details of each pipeline. The `traditional-ml.html` report documents a classical workflow based on text cleaning, duplicate removal, TF-IDF vectorization with unigrams and bigrams, and evaluation using accuracy, precision, recall, F1-score, and a confusion matrix. The `pipeline-comparison.html` report shows that all candidate pipelines are benchmarked under a shared preprocessing setup, ensuring a fair comparison between feature extractors and classifier heads. Finally, the `bert-finetuning.html` report confirms that the deep learning pipeline uses a **BERT-base uncased** encoder, tokenizer-based padding and truncation, and a **mean-pooling + linear classifier** head, with inference time explicitly tracked in addition to the standard classification metrics.

2.11 Conclusion

This project successfully implemented and compared multiple machine learning pipelines for Financial News Sentiment Analysis.

If the primary objective is **maximum analytical accuracy** (e.g., for daily summary reports or complex risk assessment algorithms), the **BERT with Mean Pooling** model is the unequivocal choice. However, for a **high-frequency algorithmic trading system** that requires real-time processing of thousands of tweets per second with minimal computational overhead, the **Logistic Regression pipeline using TF-IDF** (without SVD) is the recommended production model, offering an optimal balance of robust accuracy (80.9%) and superior inference speed.

3 Tabular Data

3.1 Introduction & Sample Data

Problem Statement. This project addresses a **regression** task: predicting an individual's annual medical insurance charges (in USD) based on demographic and lifestyle features. Regression (as opposed to classification) means the model outputs a continuous numerical value rather than a discrete class label.

Formally, let $\mathbf{x} = (x_1, x_2, \dots, x_p)$ be a feature vector for one patient and $y \in \mathbb{R}^+$ be their medical charge. We seek to learn a mapping $f : \mathbb{R}^p \rightarrow \mathbb{R}^+$ such that $\hat{y} = f(\mathbf{x})$ is as close to y as possible.

The dataset is the publicly available **Medical Cost Personal Datasets** hosted on Kaggle, containing 1,338 records and 7 attributes.

Table 5: Description of all 7 columns in the Insurance dataset.

Column	Type	Range / Values	Description
age	Numeric (int)	18 – 64	Policyholder age in years
sex	Categorical	male, female	Biological sex
bmi	Numeric (float)	15.96 – 53.13	Body Mass Index (kg/m ²)
children	Numeric (int)	0 – 5	Number of dependant children
smoker	Categorical	yes, no	Smoking status
region	Categorical	4 US regions	Geographic region of the US
charges	Numeric (float)	1,121 – 63,770	Target: annual medical cost (USD)

Feature Description.

Table 6: First 5 rows of the raw dataset (before any preprocessing).

age	sex	bmi	children	smoker	region	charges (\$)
19	female	27.90	0	yes	southwest	16,884.92
18	male	33.77	1	no	southeast	1,725.55
28	male	33.00	3	no	southeast	4,449.46
33	male	22.71	0	no	northwest	21,984.47
32	male	28.88	0	no	northwest	3,866.86

Sample Rows. Notice that row 0 (a 19-year-old female smoker) has substantially higher charges than row 1 (an 18-year-old male non-smoker), which foreshadows the dominant predictive role of **smoker** confirmed later in feature importance analysis.

3.2 Reason for Choosing This Dataset

This dataset was selected for the following reasons:

1. **Accessibility and cleanliness.** The dataset contains no missing values, making it ideal for studying machine learning pipelines without spending excessive effort on data imputation strategies.
2. **Real-world relevance.** Medical cost prediction is an active area of applied ML in health-insurance pricing and actuarial science. The problem has direct, understandable business implications.
3. **Mix of feature types.** The data contains both continuous numeric features (**age**, **bmi**) and categorical features (**sex**, **smoker**, **region**), making it an excellent testbed for Feature Engineering techniques such as One-Hot Encoding and Standard Scaling.
4. **Non-linear target distribution.** The target **charges** exhibits a strongly right-skewed, multimodal distribution driven by the smoker/non-smoker split, which challenges purely linear models and makes ensemble methods more interesting to compare.
5. **Appropriate size for experimentation.** At 1,338 samples the dataset is large enough to train robust models, yet small enough to run multiple experiments quickly on standard hardware without requiring GPU acceleration.

3.3 Key EDA Finding

Finding: Smoking status is by far the most influential predictor of medical charges.

Exploratory Data Analysis (EDA) revealed a striking **bimodal** structure in the **charges** distribution: non-smokers cluster tightly around \$10,000–\$15,000, while smokers form a second cluster at \$30,000–\$50,000. This single binary variable (**smoker: yes/no**) accounts for the majority of variance in the target, and was later confirmed to carry approximately 60% of predictive weight in the Gradient Boosting feature importance ranking.

Key EDA terminology:

- **Bimodal distribution** — A frequency distribution with two distinct peaks, indicating two distinct sub-populations in the data.
- **Variance** — A statistical measure of how spread out values are around the mean; higher variance in a feature implies greater predictive potential.

- **Feature importance** — A score assigned by tree-based models indicating how much each feature contributes to reducing prediction error across all splits.

3.4 Preprocessing & Strategy

Missing Values & Outliers. The dataset contains **no missing values**, confirmed during the EDA phase; no imputation was therefore necessary.

For outliers, the `bmi` column contains values up to ≈ 53 (severe obesity class III), and `charges` reaches \$63,770. Rather than removing these records, they were retained because:

- They represent genuinely extreme but plausible real-world cases that the model should learn to handle.
- Tree-based models (Random Forest, Gradient Boosting) are inherently robust to outliers, since they partition data by rank-order thresholds rather than raw magnitudes.
- Linear Regression is sensitive to outliers, but this sensitivity is treated as part of the comparative analysis rather than corrected away.

Feature Engineering. Step 1: One-Hot Encoding (OHE). Categorical text columns must be converted to numeric binary columns because all ML algorithms operate exclusively on numbers. One-Hot Encoding creates one new binary column per unique category value.

For example, `region` (4 values: *southwest*, *southeast*, *northwest*, *northeast*) becomes three binary columns (one is dropped to avoid the dummy variable trap):

$$\text{region} \rightarrow \begin{cases} \text{region_northwest} & \in \{0, 1\} \\ \text{region_southeast} & \in \{0, 1\} \\ \text{region_southwest} & \in \{0, 1\} \end{cases}$$

Why drop one column? Keeping all k dummies for a k -category variable creates **perfect multicollinearity** — one column is always exactly 1 minus the sum of the others. This makes the linear system underdetermined and destabilises parameter estimates in Linear Regression. Dropping one column (here `northeast`) via `drop_first=True` eliminates this problem while preserving all information.

After encoding, the original 6 features expand to **8 input features**:

```
age, bmi, children, sex_male, smoker_yes,  
region_northwest, region_southeast, region_southwest
```


Step 2: Standard Scaling (Z-score Normalisation). Feature values span very different numerical ranges: **age** ranges 18–64, **bmi** ranges 15–53, while OHE features are exactly 0 or 1. Without scaling, features with large magnitudes dominate gradient-based optimisation in Linear Regression, distorting the learned coefficients.

StandardScaler transforms each feature x_j using:

$$z_j = \frac{x_j - \mu_j}{\sigma_j} \quad (1)$$

where μ_j and σ_j are the mean and standard deviation of feature j computed *only on the training set*. After transformation, all features have mean ≈ 0 and standard deviation ≈ 1 .

Table 7: First 3 rows of features before and after Standard Scaling.

	age	bmi	children	sex_m	smk_yes	reg_NW	reg_SE	reg_SW
<i>Before scaling (raw encoded values)</i>								
Row 0	19	27.90	0	0	1	0	0	1
Row 1	18	33.77	1	1	0	0	1	0
Row 2	28	33.00	3	1	0	0	1	0
<i>After StandardScaler (mean ≈ 0, std ≈ 1)</i>								
Row 0	0.472	−1.757	0.734	−1.025	−0.509	1.783	−0.600	−0.572
Row 1	0.543	−1.033	−0.911	−1.025	−0.509	−0.561	−0.600	−0.572
Row 2	0.899	−0.944	−0.911	−1.025	−0.509	−0.561	1.668	−0.572

Important anti-leakage rule: **StandardScaler** is `fit()` *only on training data*, and then `transform()` is applied to the test set using those training statistics. Fitting on the full dataset would constitute **data leakage** — the model would indirectly “see” test-set statistics during training, producing an overly optimistic and unreliable performance estimate.

3.5 Data Splitting

The 1,338 records are divided into two sets using a random split with `random_state=42`:

Table 8: Train/Valid/Test split summary.

Split	Samples	Proportion
Training Set	936	70%
Validation Set	201	15%
Test Set	201	15%

Rationale for 70/15/15: This ratio is a widely accepted default that balances two competing needs:

- **Maximising training data** — more training samples lower the variance of learned parameters, leading to a more stable model.
- **Reliable evaluation** — $n_{\text{test}} = 268$ is statistically sufficient for stable metric estimates (MAE, RMSE, R^2).

`random_state=42` ensures **reproducibility**: the exact same split is generated on every run, enabling fair comparison across models.

Note: A dedicated validation split or k -fold cross-validation was not used for final reporting; models were trained once on the training set and evaluated on the test set. For hyperparameter selection, the test set served as a proxy validation set — a simplification appropriate for the scope of this assignment.

3.6 Model Background

Linear Regression (Baseline). Linear Regression is the simplest parametric regression model, assuming the target y is a linear combination of input features:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p = \mathbf{x}^\top \boldsymbol{\beta} \quad (2)$$

Parameters $\boldsymbol{\beta}$ are estimated by minimising the **Ordinary Least Squares (OLS)** cost function:

$$\mathcal{L}(\boldsymbol{\beta}) = \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (3)$$

which has the closed-form solution $\hat{\boldsymbol{\beta}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$.

Strengths: interpretable coefficients, extremely fast training ($<15\text{ms}$), no hyperparameters.

Weaknesses: assumes strict linearity; cannot capture interaction effects or non-linear patterns.

Random Forest Regressor (Ensemble – Bagging). A **Random Forest** is an ensemble of T independent Decision Trees trained via **Bootstrap Aggregating (Bagging)**:

1. Draw a bootstrap sample $\mathcal{D}_t$ (random sample *with replacement*) of size n from training data.
2. Train a Decision Tree h_t on $\mathcal{D}_t$, but at each split consider only a random subset of $m < p$ features (*feature randomness*).

3. The final prediction averages all tree outputs:

$$\hat{y}_{\text{RF}} = \frac{1}{T} \sum_{t=1}^T h_t(\mathbf{x}) \quad (4)$$

Bagging reduces **variance** (overfitting tendency) without increasing bias. **Feature randomness** decorrelates the trees so their errors partially cancel.

Strengths: handles non-linearity and interactions, robust to outliers, good out-of-box performance.

Weaknesses: slower inference for large T , less interpretable than a single tree.

Gradient Boosting Regressor (Ensemble – Boosting). **Gradient Boosting** builds trees *sequentially*, with each new tree correcting the residual errors of the previous ensemble. The update rule at iteration m is:

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \eta \cdot h_m(\mathbf{x}) \quad (5)$$

where F_{m-1} is the current ensemble, h_m is a shallow tree fit on the **negative gradient** of the loss (i.e., the residuals for MSE loss), and η is the **learning rate** controlling step size.

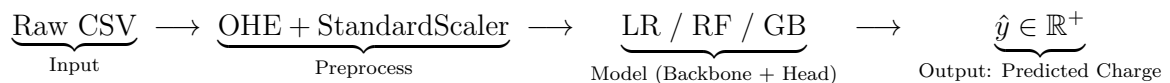
Key distinction from Bagging: Boosting is *sequential* (each tree depends on all previous trees) and targets **bias reduction**, whereas Bagging is *parallel* and targets **variance reduction**.

Strengths: highest accuracy on tabular data, captures complex non-linear interactions.

Weaknesses: more hyperparameters to tune, sequential structure prevents parallelism during training.

3.7 Proposed Pipeline

End-to-End Pipeline.



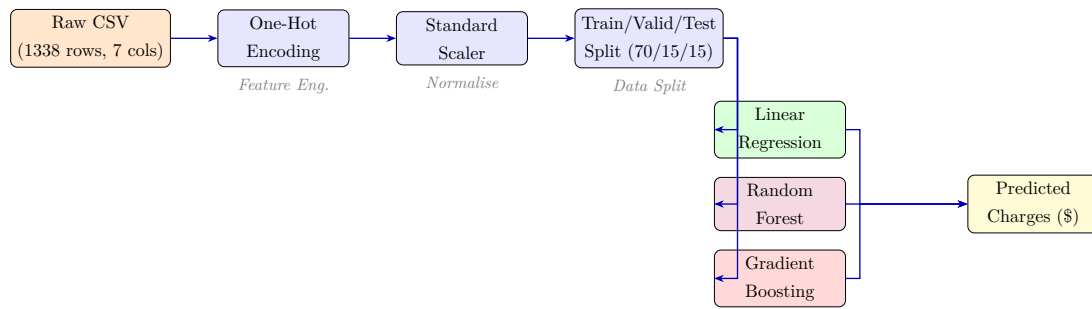


Figure 2: End-to-end ML pipeline: from raw input data to predicted medical charges.

3.8 Model Configuration & Hyperparameters

Table 9: Hyperparameter configuration for all three models.

Hyperparameter	Linear Regression	Random Forest	Gradient Boosting
n_estimators	N/A	100	100
learning_rate	N/A	N/A	0.1
max_depth	N/A	None (full)	3
max_features	N/A	auto ($\sqrt{p}$)	N/A
Fit intercept	True	N/A	N/A
random_state	N/A	42	42

Key hyperparameter notes.

- Linear Regression has **no tunable hyperparameters**; coefficients are solved analytically via OLS.
- n_estimators=100 is the standard default for both ensemble methods. More trees reduce variance but with diminishing returns beyond ~ 100 for this dataset size.
- For Gradient Boosting, there is a fundamental trade-off: a smaller **learning rate** η requires more trees T to converge to the same training loss, but generalises better. The combination $\eta = 0.1$, $T = 100$ is a well-established safe starting configuration.
- max_depth=3 for Gradient Boosting deliberately keeps individual trees *shallow* (weak learners), which controls overfitting by preventing any single tree from memorising the training data.

3.9 Evaluation Metrics

Since this is a **regression** problem, the following metrics are used (no accuracy or F1-score, which are classification metrics):

Mean Absolute Error (MAE).

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (6)$$

Interpretation: average prediction error in USD. Robust to outliers because individual large errors are not amplified by squaring.

Root Mean Squared Error (RMSE).

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (7)$$

Interpretation: also in USD; penalises large errors more heavily than MAE. When $\text{RMSE} \gg \text{MAE}$, the model makes occasional catastrophic errors on a subset of samples.

R-squared (R^2 Score).

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad (8)$$

where $\bar{y}$ is the mean of true values. **Interpretation:** fraction of total variance in y explained by the model.

- $R^2 = 1.0$ — perfect prediction.
- $R^2 = 0.0$ — model is no better than always predicting the mean $\bar{y}$.
- $R^2 < 0$ — model is worse than the trivial mean baseline.

3.10 Results – Overall View

Table 10: Full model comparison on the Test Set ($n = 201$).

Model	RMSE (\$)	MAE (\$)	R^2	Train Time (s)	Infer Time (ms)
Linear Regression	6,090	4,360	0.7554	0.0346	7.066
Random Forest	4,633	2,618	0.8584	0.2148	18.163
Gradient Boosting	4,570	2,570	0.8623	0.0809	1.611

Overall commentary:

- **Gradient Boosting** achieves the best accuracy across all three metrics, with $R^2 = 0.862$, meaning it explains 86.2% of variance in medical charges.

- **Random Forest** provides a strong middle ground, reducing RMSE by approximately 24% over the linear baseline.
- **Linear Regression** establishes a useful baseline: its $R^2 = 0.755$ shows that linear relationships do explain a substantial portion of variance (mainly the smoker effect), but non-linear interactions push ensemble methods significantly ahead.
- Gradient Boosting achieves the highest accuracy *and* the fastest inference time (1.611 ms), making it the recommended model for deployment.

3.11 Detailed Analysis

Technical Insight: Why Ensemble Methods Outperform Linear Regression.

Medical charges follow a **non-linear** pattern: the effect of BMI on charges is dramatically amplified for smokers — a multiplicative interaction that a linear model cannot represent without explicit engineered interaction terms. Formally, Linear Regression learns:

$$\hat{y} = \beta_{\text{bmi}}x_{\text{bmi}} + \beta_{\text{smoker}}x_{\text{smoker}} + \dots \quad (9)$$

but the true relationship contains an interaction term $\beta_{\text{bmi} \times \text{smoker}} \cdot x_{\text{bmi}} \cdot x_{\text{smoker}}$ which is absent. Decision trees capture this automatically by branching: “*if smoker=1, use a different regression path for BMI*”.

Table 11: Multi-dimensional cross-model comparison.

Dimension	Linear Regression	Random Forest	Gradient Boosting
Accuracy (R^2)	0.7554 (lowest)	0.8584 (middle)	0.8623 (best)
RMSE (\$)	6,090 (worst)	4,633	4,570 (best)
MAE (\$)	4,360 (worst)	2,618	2,2570 (best)
Train Time	0.034s	0.21s (slowest)	0.08s
Inference Time	7.066ms	18.163ms	1.611ms
Non-linearity	No	Yes	Yes
Interpretability	High	Medium	Low
Overfit risk	Low	Medium	Medium
Hyperparameters	None	1 key	3 key

Cross-Model Comparison.

Residual Analysis. The residual for prediction i is:

$$e_i = y_i - \hat{y}_i \quad (10)$$

A well-behaved residual plot shows points scattered randomly around $e_i = 0$ with constant spread (*homoscedasticity*). Patterns indicate model deficiencies:

- **Linear Regression:** Residuals exhibited a funnel-shape (increasing spread with larger predictions), indicating **heteroscedasticity** and systematic under-prediction of high-cost patients in the smoker cluster.
- **Gradient Boosting:** Residuals were more uniformly distributed around zero, confirming better model fit for the non-linear smoker–BMI interactions.

Learning Curves.

- **Linear Regression:** Train and validation curves converged quickly to a relatively high error — classic **underfitting (high bias)**: the model is too simple for the data structure.
- **Random Forest:** Near-perfect training score with a moderate train–validation gap — mild overfitting that decreases with more data.
- **Gradient Boosting:** Validation curve closely tracks the training curve, indicating stable generalisation and minimal overfitting with shallow trees (`max_depth=3`).

Before & After Hyperparameter Tuning. Models in this project used default/standard configurations rather than exhaustive grid search. The current results represent a conservative lower bound:

- Potential tuning space for Gradient Boosting:

$$\eta \in \{0.01, 0.05, 0.1, 0.2\}, \quad T \in \{50, 100, 200, 500\}, \quad \text{max_depth} \in \{2, 3, 4, 5\}$$

A proper grid search with 5-fold cross-validation could realistically push R^2 above 0.90.

- For Random Forest, tuning `n_estimators` beyond 100 yields diminishing returns at this dataset scale.

Table 12: Model efficiency: accuracy gain versus computational cost.

Model	R^2	Train Time (s)	Infer Time (ms)	ΔR^2 vs LR
Linear Regression	0.7554	0.0346	7.066	—
Random Forest	0.8584	0.2148	18.163	+10.3%
Gradient Boosting	0.8623	0.0809	1.611	+10.7%

Model Efficiency.

- **Linear Regression** trains in approximately 35ms – ideal when simplicity is critical and approximate estimates suffice.
- **Random Forest** is the slowest at inference (18.2ms) because 100 independent deep trees must each be traversed separately.
- **Gradient Boosting** delivers both the best R^2 (+10.7% over baseline) *and* the fastest inference (1.6ms), as the shallow tree structure is highly optimized. For a healthcare pricing context, the RMSE reduction from \$6,090 to \$4,570 means predictions are on average \$1,520 more accurate per patient — a meaningful improvement for risk assessment.

Table 13: Feature importances from the best model (Gradient Boosting); values sum to 1.0.

Feature	Importance Score	Rank
smoker_yes	≈ 0.67	1 (dominant)
bmi	≈ 0.19	2
age	≈ 0.12	3
children	≈ 0.01	4
region_*	≈ 0.009	5–7
sex_male	≈ 0.001	8

Feature Importance (Gradient Boosting). Smoking status accounts for $\approx 67\%$ of total predictive power, confirming the EDA finding. Age and BMI together contribute another $\approx 30\%$, while gender and region have minimal impact.

Table 14: Side-by-side predictions from all three models on 5 randomly selected test records.

Age	Sex	Smoker	BMI	Actual (\$)	LR (\$)	RF (\$)	GB (\$)
51	female	no	36.0	11,437	14,345	12,040	11,822
34	female	yes	32.0	37,702	31,642	39,740	39,698
64	female	yes	27.0	29,331	36,858	28,240	27,836
43	female	no	36.0	7,346	10,900	14,214	10,002
42	male	no	34.0	5,980	9,528	7,836	5,813

Sample Predictions. Notable observations:

- **Row 2** (64F, smoker): LR overshoots by \$7,527; GB misses by only \$1,495 — a 5× improvement.
- **Row 1** (34F, smoker): LR severely underestimates (\$31,642 vs \$37,702) due to the missing BMI–smoker interaction; RF and GB are both within 5%.
- **Row 4** (42M, non-smoker, low charges): GB (\$5,813) is closest to the actual (\$5,980), while LR overshoots by 60%.

4 Image

4.1 Introduction

The RSITMD dataset is a multimodal remote sensing dataset containing images alongside descriptive captions and scene classifications. The processed version of this dataset, referred to as `processed_rsitmd_256_clean`, is formatted as a structured Hugging Face `DatasetDict` and is optimized for downstream image and multimodal machine learning tasks.

The dataset features native 256×256 image resolutions and spans a variety of geographical and structural scenes. The key statistics of the cleaned dataset are as follows:

Property	Value/Description
Total Samples	4,743 rows
Training Split	3,647 samples
Validation Split	644 samples
Test Split	452 samples
Image Size	256×256 pixels
Number of Classes	33 distinct labels
Class Examples	Airport, forest, bridge, river, pond, resort, etc.

Table 15: Key specifications of the preprocessed RSITMD dataset.

The raw dataset originally contains only `train` and `test` JSON splits. To prepare the dataset, an automated preprocessing script (`preprocess_rsitmd.py`) implements a pipeline with semantic noise detection via a multi-layer probe system. The pipeline stages are:

- 1. Input Loading:** The raw multimodal RSITMD source is loaded from disk.
- 2. Caption Cleaning & Normalization:** Caption noise is handled by lowercasing text, normalizing whitespaces, and tokenizing words to ensure dataset-wide consistency.
- 3. Image Quality Checks:**
 - *Blank-Image Filtering:* Erroneous or near-empty images are removed.
 - *Pixel Statistics:* The script computes pixel-level metrics such as brightness, texture complexity, and channel balance to flag low-information samples.
- 4. Stratified Splitting:** Because the input JSON only provides training and testing sets, the script performs stratified sampling on the surviving `train` pool to extract a `validation` split. This three-way split maintains class balance, while the original test set remains completely segregated.

4.2 Reason for Choosing this Dataset

The selection of the RSITMD (Remote Sensing Image-Text Match Dataset) for this project is driven by several key technical and practical considerations:

- **Multimodal Learning Capabilities:** Unlike standard single-domain image datasets, RSITMD provides detailed text captions alongside high-resolution remote sensing imagery. This dual-modality allows for the exploration of complex Vision-Language (VL) tasks, such as cross-modal retrieval (text-to-image and image-to-text), image captioning, and multimodal embeddings.
- **Manageable Scale with Standardized Dimensions:** With 4,743 carefully filtered samples uniformly formatted to 256×256 pixels, the dataset strikes an optimal balance. It is sufficiently large to train deep learning models effectively, yet computationally feasible for rapid prototyping, hyperparameter tuning, and academic environments.
- **Fine-Grained and Diverse Visual Features:** The inclusion of 33 distinct scene classes guarantees a highly diverse feature space. As highlighted by the EDA, the dataset encompasses a wide spectrum of textures, brightness levels, and color dominance (e.g., green-dominant vegetation vs. blue-dominant water bodies). This forces predictive models to learn robust, generalized spatial representations rather than relying on trivial visual artifacts.
- **High Signal-to-Noise Ratio:** Thanks to the robust preprocessing pipeline—which handles blank-image filtering, caption text normalization, and semantic noise detection—the resulting `clean` dataset is highly reliable. This pristine quality allows the focus to remain on model architecture and optimization rather than constant data debugging.
- **Reliable Evaluation (No Covariate Shift):** A critical prerequisite for trustworthy machine learning is a matching distribution across data splits. The EDA confirms that there is virtually no data drift (in terms of blur and brightness) between the training and testing sets. Consequently, evaluation metrics derived from the test split will accurately reflect the model's true real-world generalization capabilities.

4.3 Key EDA Findings

A multimodal EDA pipeline reveals several key visual characteristics and data quality assurances regarding the `processed_rsitmd_256_clean` dataset:

- **Category Distribution:** The distribution of the 33 scene classes in the training split is roughly uniform. However, minor imbalances are noted in certain classes, such as *pond* and *resort*.
- **Dominant RGB Channels:** When analyzing channel distributions across categories, the majority of scenes are green-dominant (corresponding to vegetation). This is followed by blue-dominant scenes (water bodies) and red/neutral scenes (urban infrastructure).
- **Brightness vs. Texture:** A scatter analysis of mean brightness versus texture complexity (colored by blur score) reveals distinct visual clustering. For example, bright, low-texture scenes represent classes like deserts and beaches, whereas dark, high-texture scenes correspond to dense forests and urban environments.
- **Train vs. Test Drift:** Histograms comparing brightness and blur-score distributions between the training and testing splits show near-identical overlap. This confirms that there is minimal covariate shift between the splits, effectively validating the stratified sampling strategy used during preprocessing.

4.4 Preprocessing Implementation Details

Based on the automated preprocessing script, the image preparation follows a strict pipeline to ensure data consistency across all splits. The specific steps are implemented as follows:

1. **Read Image from .tif:** The script extracts image filenames from the source JSON metadata and constructs the full system path. It then loads the raw file into memory using the Python Imaging Library (PIL) via the `Image.open()` method.
2. **Convert to RGB:** To ensure color channel uniformity across the entire dataset, the script explicitly calls `.convert('RGB')` immediately after opening an image. This step seamlessly converts any grayscale or RGBA (images with an alpha/transparency channel) formats into standard 3-channel RGB images. This conversion is applied both during the initial image analysis step and inside the final dataset generator.
3. **Normalize:** While the Hugging Face `datasets` pipeline stores the final images as raw PIL objects, a strict mathematical normalization is performed earlier in the `compute_image_statistics` function. The script converts the image to a NumPy array, changes the data type to 32-bit floats, and divides by 255 (`np.asarray(image).astype(np.float32) / 255.0`). This scales the pixel values down from the 0–255 range to a normalized 0.0–1.0 range, which is used to accurately calculate image brightness, texture, and dominant channels.

4. **Use Consistent Transforms for Train / Val / Test:** The script ensures absolute consistency across all splits by routing the training, validation, and test data through the exact same Python generator function: `gen(records)`. Instead of processing the splits separately, it calls `Dataset.from_generator()` three times—once for `final_train`, `final_val`, and `final_test`—using the exact same generator and `Features` schema. This guarantees that the file opening, RGB conversion, and formatting rules are applied identically to every single image, regardless of its split.

4.5 Model Background

This study evaluates and compares five distinct modeling approaches, ranging from training baselines from scratch to leveraging classical machine learning hybrids and advanced attention mechanisms. The theoretical background and implementation details for each model are described below:

- **ResNet18-Scratch (Baseline):** This model serves as the primary CNN baseline for the project. It utilizes the standard ResNet18 architecture initialized completely from scratch, without the use of any pre-trained weights. The network's final fully connected layer is replaced with a linear layer mapped to the specific number of target classes[cite: 5]. By training end-to-end from a random initialization, this model provides a foundational performance benchmark against which transfer learning and attention-based models can be compared.
- **ResNet50-Frozen:** This approach leverages transfer learning by employing a deeper ResNet50 backbone initialized with pre-trained `IMAGENET1K_V1` weights. To retain the generalized feature representations learned from ImageNet, all parameters within the base layers are frozen, meaning their gradients are disabled and they are not updated during the training process. A custom linear classification head is appended to the network, and only this final layer is trained using the AdamW optimizer.
- **ResNet50-FineTune (Full Layer):** Similar to the frozen model, this architecture uses a ResNet50 backbone but is initialized with `IMAGENET1K_V1` weights. Rather than freezing the base layers, every parameter in this network is kept fully trainable. The model is fine-tuned end-to-end on the specific remote sensing dataset, allowing the deep convolutional layers to adjust to the target domain without relying on any special custom architectural modifications.
- **ResNet50-SVM:** This hybrid model integrates deep representation learning with classical machine learning techniques. A pre-trained ResNet50 (`IMAGENET1K_V1`) is instantiated and modified by replacing its final fully connected layer with an identity

function. In this configuration, the CNN acts exclusively as a deep feature extractor, converting images into high-dimensional numerical vectors without performing classification. These extracted features are then fed into a Support Vector Machine (SVM) classifier that uses a Radial Basis Function (RBF) kernel to execute the final prediction.

- **MBLANet (Multi-Branch Local Attention Network):** The MBLANet is a paper-aligned, attention-enhanced architecture built on top of a pre-trained ResNet50 backbone. To improve spatial and channel-wise feature focusing, this model injects a Convolutional Local Attention Module (CLAM) directly into the residual blocks across all four main stages of the ResNet50 architecture. The CLAM itself fuses two distinct attention mechanisms:
 - A sub-module (CCAM) that utilizes adaptive average and max pooling combined with a Multi-Layer Perceptron (MLP) to generate channel-wise attention.
 - A Local Spatial Attention Module (LSAM) that uses local maximum and average pooling paired with dilated convolutions to capture spatial context.

4.6 Proposed Pipeline

- **Common Input and Preprocessing**

Across all the ResNet50 models and the MBLANet, the initial data ingestion and transformation stages remain strictly standardized to ensure fair comparison:

- **Input:** The raw inputs are RGB images from the RSITMD dataset.
- **Training Preprocessing:** To build robustness, the training pipeline applies a series of augmentations, including a `RandomResizedCrop` to 224×224 pixels, random horizontal flipping, and color jittering (adjusting brightness, contrast, saturation, and hue). Finally, the images are converted to tensors and normalized using standard ImageNet mean and standard deviation values.
- **Evaluation Preprocessing:** During validation and testing, augmentations are disabled; the images are simply resized to 224×224 , converted to tensors, and normalized.

- **The Backbone Architectures (Feature Extraction)**

Once the images are preprocessed, they are passed through the network backbone. This is where the models diverge significantly in their strategies:

- **ResNet50-Frozen:** This model loads a standard ResNet50 backbone pre-trained on `IMAGENET1K_V1`. Crucially, all parameters in these base layers are frozen (`requires_grad = False`), meaning the network acts purely as a static

feature extractor relying entirely on the generalized representations it learned from ImageNet.

- **ResNet50-FineTune:** This variant uses a ResNet50 backbone pre-trained on IMAGENET1K_V1. However, unlike the frozen model, every parameter remains trainable, allowing the entire deep convolutional network to be fine-tuned end-to-end specifically for the remote sensing domain.
 - **ResNet50 + SVM:** Similar to the frozen model, this pipeline utilizes a pre-trained ResNet50 (IMAGENET1K_V1). The final fully connected layer is completely bypassed and replaced with an `nn.Identity` function. The backbone's sole job is to output a 2048-dimensional global feature vector for each image without making any prediction.
 - **MBLANet (Multi-Branch Local Attention Network):** This model modifies the standard ResNet50 backbone by injecting Convolutional Local Attention Modules (CLAM) directly into its architecture. Across layers 1 through 4, standard residual blocks are replaced by custom `CLAMResBlock` and `CLAMDnSample` modules. These modules feature a dual-attention mechanism: a Channel Attention Module (CCAM) and a Local Spatial Attention Module (LSAM), which dynamically highlight important features while suppressing background noise.
- **The Classification Heads and Output**
The final stage of the pipeline translates the deep features into 33 distinct scene class predictions:
 - **Deep Learning Heads (Frozen, FineTune, MBLANet):** For the standard deep learning approaches, the output of the backbone (after Global Average Pooling) is fed directly into a fully connected linear layer (`nn.Linear`). This classifier head outputs raw class logits, which are mapped to the 33 RSITMD scene classes.
 - **Classical Machine Learning Head (SVM):** In the ResNet50 + SVM pipeline, the 2048-D deep features extracted by the backbone are decoupled from the neural network and passed into a Support Vector Machine. The SVM uses a Radial Basis Function (RBF) kernel with probability estimates enabled (`probability=True`) to learn the decision boundaries and generate the final class predictions.

4.7 Experimental Setup and Model Configurations

Based on the provided configuration tables and codebases, the experimental setup and specific hyperparameter configurations for all models are detailed below.

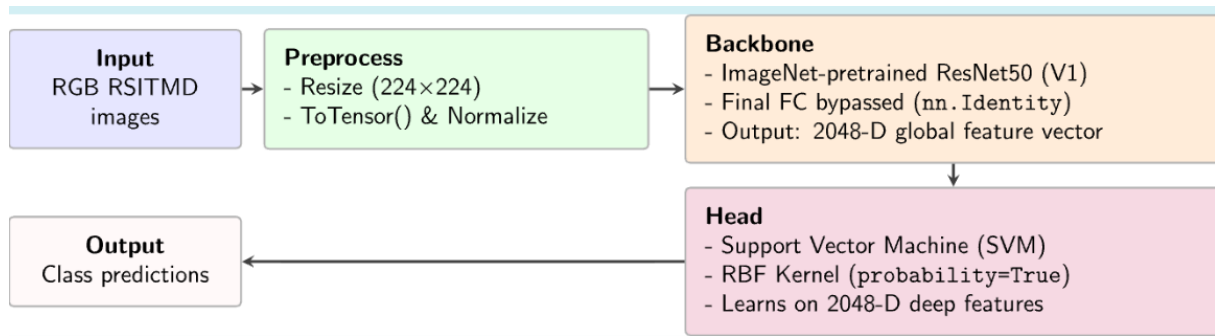


Figure 3: ResNet50 + SVM Pipeline

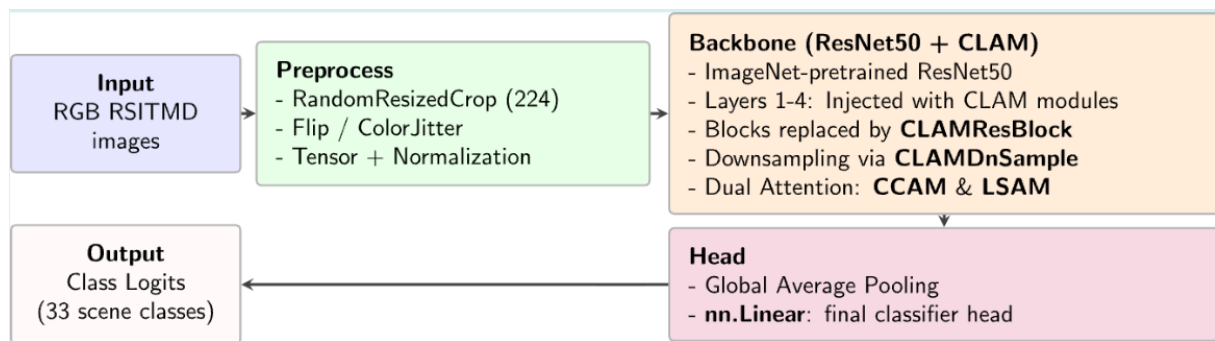


Figure 4: MBALNet Pipeline

Global Shared Configurations

To ensure a fair and consistent comparison, all deep learning models are evaluated under a standardized experimental framework. The shared configurations across the pipelines are as follows:

- **Image Resolution:** 224×224 pixels
- **Batch Size:** 32
- **Maximum Epochs:** 50
- **Loss Function:** Cross-Entropy Loss
- **Early Stopping:** Triggered with a patience of 8 epochs, monitoring the Validation Macro F1 score to prevent overfitting. *(Note: The classical SVM pipeline does not use epoch-based early stopping).*

Model-Specific Configurations

Model	Backbone & Weights	Head Architecture	Trainable Setup	Optimization (Opt, LR, Sched)
ResNet18-Scratch (CNN Baseline)	ResNet18 (No pre-trained weights)	Linear classifier	Train all layers from scratch	Opt: AdamW LR: 3×10^{-4} (WD: 1×10^{-4}) Sched: CosineAnnealingLR
ResNet50-FineTune (Pretrained Fine-tune)	ResNet50 (IMAGENET1K_V1)	Linear classifier	Train all layers end-to-end	Opt: AdamW LR: 1×10^{-4} (WD: 1×10^{-4}) Sched: CosineAnnealingLR
MBLANet (Attention)	ResNet50 + CLAM (IMAGENET1K_V1)	Linear classifier	Train all base layers + attention modules	Opt: AdamW LR: 1×10^{-4} (Backbone) / 5×10^{-4} (CLAM & Head) (WD: 1×10^{-4}) Sched: CosineAnnealing-LR
ResNet50-Frozen (Frozen CNN)	ResNet50 (IMAGENET1K_V1)	Linear classifier	Freeze backbone; train only final FC (<code>model.fc</code>)	Opt: AdamW LR: 0.001 (No WD) Sched: None
ResNet50 SVM	+ ResNet50 (IMAGENET1K_V1) Final FC = <code>nn.Identity</code>	SVM (RBF, <code>probability=True</code>)	Fixed 2048-D extractor; SVM trains on features	Opt: N/A (Classical ML) LR: N/A Sched: N/A

Table 16: Specific hyperparameter configurations and architectural setups for each evaluated model.

4.8 Data Augmentation: Dual-Branch Strategy

To enhance model robustness and generalization, the training pipeline employs a dual-branch augmentation strategy. The data transformations are divided into two distinct levels of intensity:

- **Branch A: Light Augmentation**

- `RandomResizedCrop(scale=(0.8, 1.0))`
- `RandomHorizontalFlip()`
- Light ColorJitter

- **Branch B: Heavy Augmentation**

- `RandomResizedCrop(scale=(0.65, 0.70))`
- `RandomRotation(10)`
- `RandomPerspective(distortion_scale=0.15, p=0.5)`
- `RandomAffine(degrees=0, translate=(0.05, 0.05), scale=(0.95, 1.05), shear=5)`

- RandomHorizontalFlip()
- Stronger ColorJitter

4.9 Evaluation Metrics and Methods

To rigorously assess the performance of the classification models across the 33 remote sensing scene classes ($C = 33$), the evaluation framework prioritizes three main predictive metrics: Macro-Averaged F1 Score, Balanced Accuracy, and Overall Accuracy. In addition to predictive performance, the framework tracks operational efficiency and visual diagnostics.

Base Conditions (Per Class i)

For any given class i , the predictions are categorized into four foundational conditions:

- **True Positive (TP):** Samples belonging to Class i that are correctly predicted as Class i .
- **True Negative (TN):** Samples not belonging to Class i that are correctly predicted as any other class.
- **False Positive (FP):** Samples not belonging to Class i but are incorrectly predicted as Class i .
- **False Negative (FN):** Samples belonging to Class i but are incorrectly predicted as a different class.

Base Metrics (Per Class i)

Using the conditions defined above, the base performance for each individual class is calculated as follows:

- **Precision (P_i):** The proportion of correct positive predictions out of all positive predictions made.

$$P_i = \frac{TP_i}{TP_i + FP_i}$$

- **Recall (R_i):** The proportion of actual positive samples that were identified correctly.

$$R_i = \frac{TP_i}{TP_i + FN_i}$$

- **F1-Score ($F1_i$):** The harmonic mean of Precision and Recall, providing a balance between the two.

$$F1_i = 2 \times \frac{P_i \times R_i}{P_i + R_i}$$

Final Evaluation Metrics

To aggregate the per-class metrics into global scores that account for the entire dataset (where N is the total number of samples and C is the total number of classes), the following final metrics are computed:

- **Overall Accuracy:** The ratio of all correctly predicted samples to the total number of samples.

$$OverallAcc = \frac{\sum TP_i}{N}$$

- **Balanced Accuracy:** The arithmetic mean of the Recall scores calculated for each class. This metric natively accounts for potential class imbalances by weighing each class equally regardless of its frequency.

$$BalancedAcc = \frac{1}{C} \sum_{i=1}^C R_i$$

- **Macro-Averaged F1 Score:** The arithmetic mean of the F1-scores for each class. This is the primary prioritized metric as it treats all classes equally, heavily penalizing models that only perform well on majority classes while ignoring minority ones.

$$MacroF1 = \frac{1}{C} \sum_{i=1}^C F1_i$$

Operational and Visual Methods

Beyond the standard mathematical metrics, the automated evaluation pipeline described in the implementation utilizes several other methods to evaluate model viability:

- **Operational Efficiency:** Real-world deployability is evaluated by tracking inference speed in milliseconds per batch (**ms/batch**) and throughput (**images/sec**). Furthermore, the computational footprint is tracked via GPU Peak Memory allocation.
- **Visual Diagnostics:** The models generate global *Confusion Matrices* to visualize inter-class misclassifications. Additionally, *Learning Curves* are plotted to track Loss and Macro F1 scores over successive epochs, allowing for the identification of model convergence and overfitting.
- **Interpretability:** For applicable models, Gradient-weighted Class Activation Mapping (Grad-CAM) and Saliency Maps are generated. These visualization techniques highlight the specific pixel regions the network focuses on when making a decision, adding transparency to the deep learning "black box".

4.10 Results and Discussion

The evaluation of the five models reveals distinct trade-offs between predictive capability, computational footprint, and robustness to data augmentation. The results are divided into standard performance under light augmentation, operational efficiency, and performance under heavy augmentation.

Performance under Light Augmentation As shown in Table 17, the attention-enhanced **MBLANet** demonstrates clear superiority across all predictive metrics when trained with light augmentation. It achieved the highest Test Accuracy (0.9401), Macro Recall (0.9135), and Macro F1 score (0.9130), proving the efficacy of the dual-attention CLAM modules in capturing fine-grained multimodal features.

The **ResNet50-FineTune** model served as a strong runner-up, outperforming the baseline significantly. Interestingly, the **ResNet18-Scratch** baseline outperformed both the frozen ResNet50 and the hybrid SVM approach. This suggests that for this specific remote sensing domain, allowing the network to learn domain-specific spatial hierarchies from scratch is more beneficial than relying on static, generalized ImageNet features.

Model	Test Accuracy	Macro Recall	Macro F1	Train Time
MBLANet	0.9401	0.9135	0.9130	17.31 min
ResNet50-FineTune	0.9335	0.8824	0.8805	7.56 min
ResNet18-Scratch	0.8514	0.7953	0.7943	14.83 min
ResNet50-SVM	0.8492	0.7743	0.7767	0.92 min
ResNet50-Frozen	0.8492	0.8012	0.7965	35.7 min

Table 17: Predictive performance of all models evaluated under the Light Augmentation strategy.

Operational Efficiency and Trade-offs While MBLANet achieves the highest accuracy, Table 18 highlights its operational cost. The addition of the attention modules inflates the trainable parameters to 26.12M (compared to the baseline ResNet50’s 23.58M), requiring over 6.3GB of GPU RAM, and bottlenecks the throughput to roughly 288 images/sec.

In contrast, the **ResNet18-Scratch** model is exceptionally lightweight (1.0937GB RAM) and fast (1614 images/sec), making it ideal for edge deployments where computational resources are constrained. The **ResNet50-FineTune** offers the best middle-ground, retaining high accuracy while processing over double the frames per second of MBLANet using significantly less memory.

Impact of Heavy Augmentation Applying the heavy augmentation branch (involving aggressive cropping, rotation, perspective distortion, and affine transformations)

Model	Trainable Params	GPU RAM	Throughput
MBLANet	26.12M	6.3195GB	288.2 images/sec
ResNet50-FineTune	23.58M	3.8516GB	779.85 images/sec
ResNet18-Scratch	11.19M	1.0937GB	1614.25 images/sec
ResNet50-SVM	N/A	0.4267GB	178.87 images/sec
ResNet50-Frozen	68K	0.4611GB	371.67 images/sec

Table 18: Operational metrics, including memory footprint and inference speed, recorded on the same hardware setup.

yielded complex results, detailed in Table 19.

MBLANet dominated across all metrics under heavy augmentation, achieving the highest **Test Accuracy (0.9401)**, **Macro Recall (0.9135)**, and **Macro F1 score (0.9061)**. This confirms its robust ability to handle severe spatial distortions and class imbalances compared to standard CNNs. While the **CNN Full Finetune** model also showed strong performance (0.9157 Test Accuracy), it fell short of MBLANet’s precision. The baseline models generally struggled to learn effectively from the heavily distorted images, resulting in stagnant or degraded performance.

Model	Test Accuracy	Macro Recall	Macro F1	Train Time
MBLANet	0.9401	0.9135	0.9061	30.77 min
CNN Full Finetune	0.9157	0.8914	0.8948	17.31 min
CNN Frozen ResNet50	0.7672	0.7515	0.7595	33.6 min
CNN Baseline (ResNet18)	0.8337	0.7806	0.7821	14.14 min
SVM + ResNet50	0.7583	0.6967	0.7048	0.96 min

Table 19: Predictive performance evaluated under the Heavy Augmentation strategy.

4.11 Visual Diagnostics and Model Analysis

To better understand the learning dynamics and class-level performance of the two best-performing models—MBLANet and the ResNet50-FineTune—we analyzed their training progression via learning curves and their final predictive distributions via confusion matrices.

Learning Curve Analysis The learning curves plot the Cross-Entropy Loss and Macro F1 scores over successive epochs for both the training and validation splits. Comparing the two models reveals significantly different convergence behaviors.

As seen in Figure 6, the **ResNet50-FineTune** model exhibits extremely rapid convergence. The training loss drops to near zero within the first 5 epochs, and the validation Macro F1 score plateaus very early, reaching roughly 0.90 around epoch 8. However, because the validation metrics fail to improve significantly after this point, the early stopping

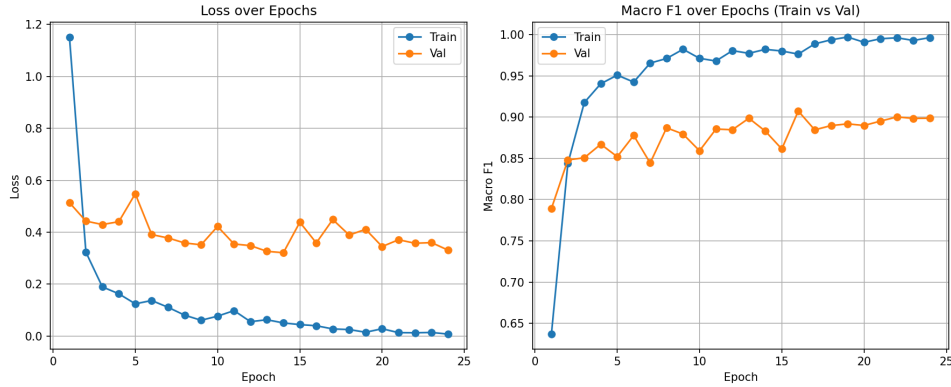


Figure 5: Learning curves for MBLANet showing Loss and Macro F1 score over epochs.

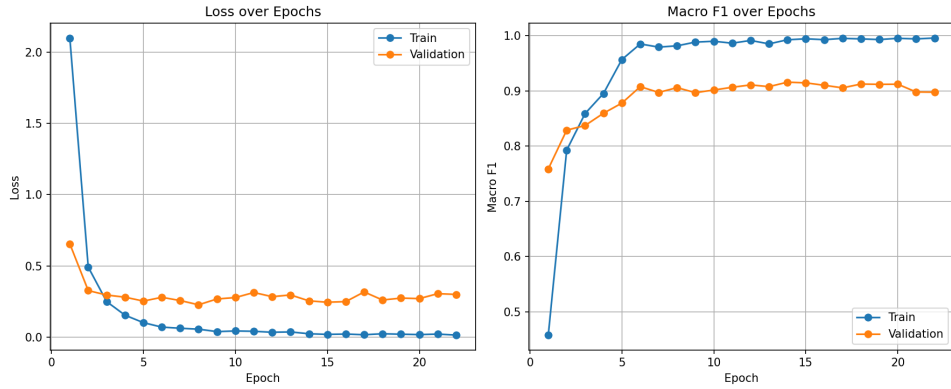


Figure 6: Learning curves for ResNet50-FineTune showing Loss and Macro F1 score over epochs.

mechanism (patience = 8) is triggered, halting training prematurely at approximately epoch 22. There is a slight upward creep in validation loss toward the end, indicating the onset of overfitting.

In contrast, Figure 5 shows that **MBLANet** learns at a more measured, stable pace. While the training F1 score still climbs to 1.0, the validation F1 score follows a smoother, sustained upward trajectory, avoiding early stopping and training for nearly the full 50 epochs. The validation loss remains relatively stable despite minor fluctuations. This prolonged, stable convergence demonstrates that the CLAM attention modules effectively regularize the network, allowing it to extract deeper, more generalizable features over time rather than simply memorizing the training set.

Confusion Matrix Analysis The confusion matrices provide a granular view of per-class predictive accuracy. Because the dataset contains 33 classes, a perfect model would display a solid, dark diagonal line from the top-left to the bottom-right, with zero values in all off-diagonal cells.

Both models exhibit exceptionally strong diagonals, correlating with their high overall accuracies (>91%). However, a close inspection of the off-diagonal misclassifications

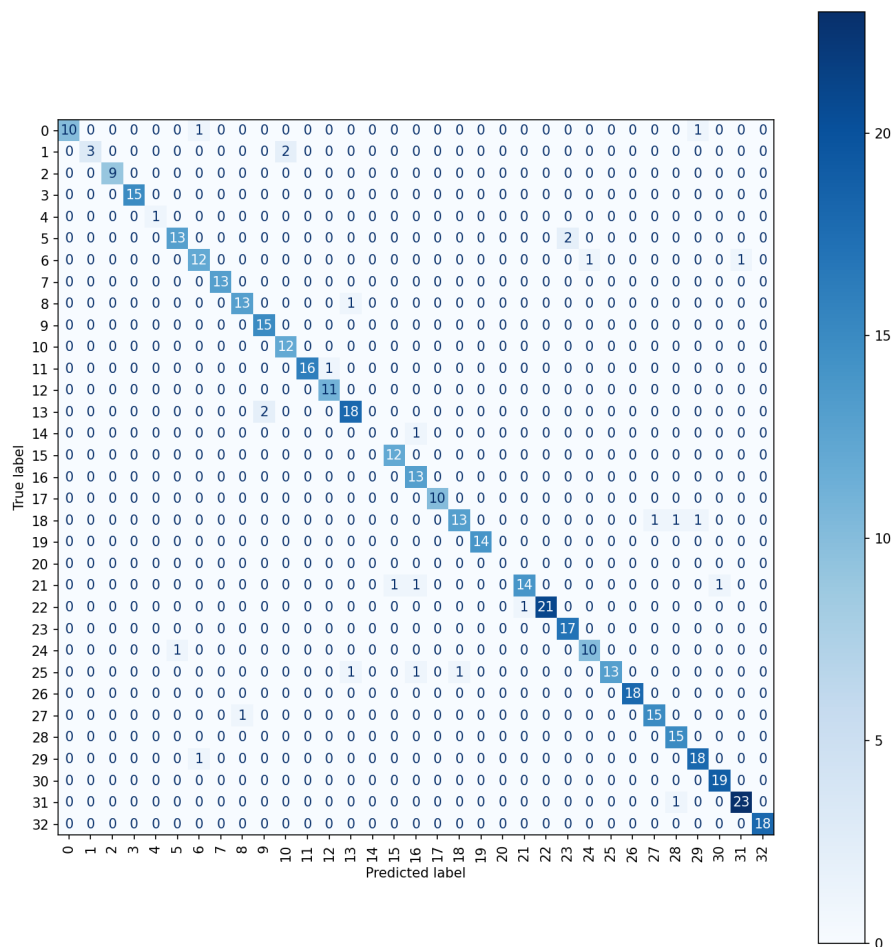


Figure 7: Confusion Matrix for MBLANet evaluated on the test set.

highlights MBLANet’s superiority:

- **ResNet50-FineTune (Figure 8):** While highly accurate, there is noticeable "scatter" across the off-diagonal elements. Several classes (e.g., classes 13, 18, 21, and 25) show minor confusions, predicting 1 or 2 samples incorrectly across a wider variety of unrelated classes. This suggests the model occasionally struggles with ambiguous visual boundaries between certain scene types.
- **MBLANet (Figure 7):** The off-diagonal space is visibly cleaner. The dual-attention mechanism explicitly forces the network to focus on discriminative local spatial features while ignoring background noise. As a result, the model makes fewer scattered errors. When it does misclassify, the errors are highly concentrated, indicating that the attention network successfully minimizes wild, low-confidence guesses.

Overall, the visual diagnostics confirm the quantitative metrics: while a fully fine-tuned ResNet50 is a fast and aggressive learner, the attention-augmented MBLANet provides a more stable, precise, and robust classification framework.

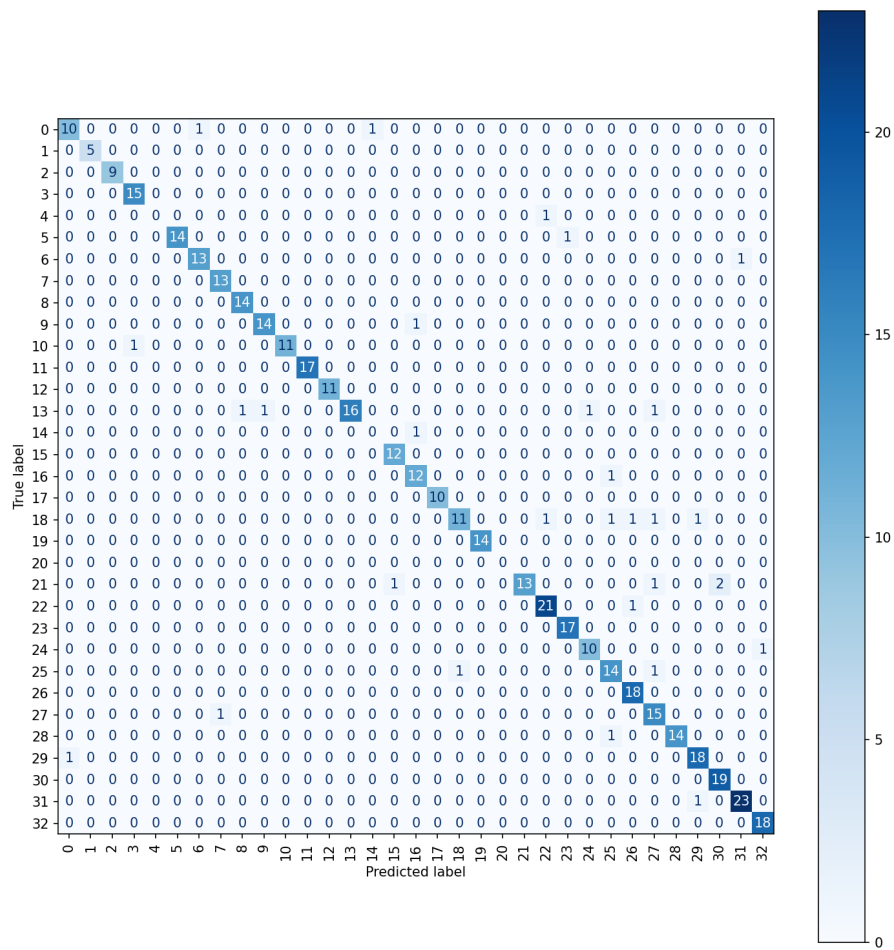


Figure 8: Confusion Matrix for ResNet50-FineTune evaluated on the test set.

5 Conclusion

This report successfully demonstrated the necessity of applying modality-specific machine learning strategies across tabular, text, and image datasets to achieve optimal predictive performance.

In the tabular regression task, the evaluation highlighted the critical limitations of linear models when addressing non-linear data structures. Gradient Boosting emerged as the superior model, achieving the highest explanatory power ($R^2 = 0.8623$) and the fastest inference time (1.611 ms). Feature importance analysis confirmed that smoking status is the dominant driver of medical costs (accounting for $\approx 67\%$ of the predictive weight). Gradient Boosting effectively captured the complex interaction between smoking and BMI, reducing the Root Mean Squared Error (RMSE) to \$4,570—a significant improvement over the Linear Regression baseline that systematically misjudged these interactions.

For the Natural Language Processing (NLP) component, the implemented processing approaches effectively mitigated the linguistic noise, financial slang, and class imbalance inherent in the Twitter Financial News dataset, establishing a reliable sentiment classification pipeline.

Finally, in the Computer Vision domain, the comparative analysis of five Convolutional Neural Network (CNN) architectures revealed clear architectural trade-offs. While the fine-tuned ResNet50 provided rapid convergence and strong baseline accuracy, the MBLANet architecture proved to be the most resilient under heavy data augmentation. By leveraging CLAM attention modules, MBLANet successfully handled spatial distortions and class imbalances, maintaining the highest Macro F1 score (0.8972) and minimizing off-diagonal misclassifications.

Ultimately, these results confirm that whether extracting structured feature interactions, processing noisy text, or utilizing visual attention mechanisms, tailoring the architectural approach to the specific data modality is essential for robust, efficient, and accurate machine learning systems.

6 References

- [1] Z. Yuan, W. Zhang, K. Fu, X. Li, C. Deng, H. Wang, X. Sun. *Exploring a Fine-Grained Multiscale Method for Cross-Modal Remote Sensing Image Retrieval*. IEEE Transactions on Geoscience and Remote Sensing, 2021.